

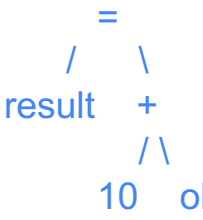
Programming Languages

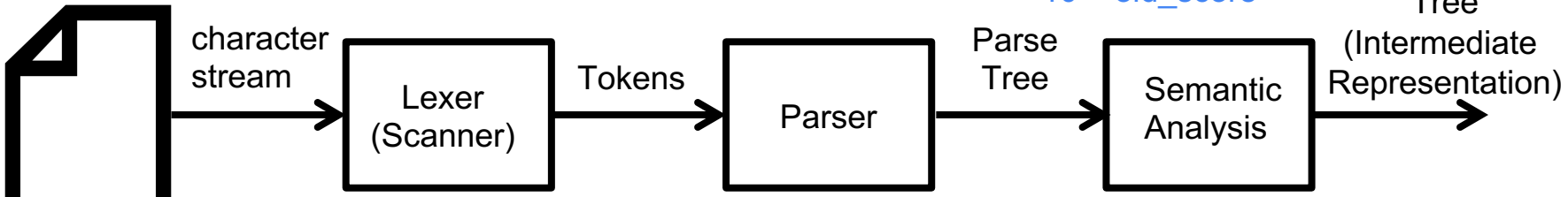
8. Basic Semantics
(Chapter 7)

Ji Eun Kim

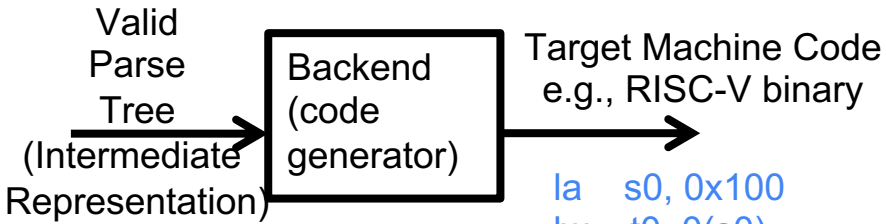
Fall 2026

Overview of Program Interpretation





- Have result and old_score been declared?
- Are 10 and old_score types that can be added together (e.g., both are numbers)?
- Can the result of the addition be assigned to the result variable?
- ...




```
la   s0, 0x100
lw   t0, 0(s0)
addi t1, t0, 10
la   s1, 0x104
sw   t1, 0(s1)
```

Topics

- Attributes, binding, and semantic functions
- Declarations, blocks, and scope
- Symbol table
- Name resolution and overloading
- Allocation, lifetimes, and the environment

Specifying Semantics (1/2)

- Specifying semantics is more difficult than specifying syntax
- Several ways to specify semantics:
  1. Language reference manual:
     - Most common approach to specify semantics
     - Sffers from a lack of precision inherent in natural language descriptions
     - May have omissions and ambiguities
  2. Defining a translator
  3. Formal definition

Specifying Semantics (2/2)

2. Defining a translator
    - Behavioral Experimentation: answers questions about language rules through runtime experimentation .
    - Cannot predict general program behavior in advance.
    - Vulnerable to unintentional bugs, machine dependencies, and lack of portability
3. Formal Definition
    - Mathematical Rigor: uses precise mathematical techniques, though highly complex and abstract.
    - Denotational Semantics: uses a series of mathematical functions to describe translation and execution, regarded as one of the best formal method

Attributes, Binding and Semantic Functions

Name and Attributes

- **Names** (or **identifiers, 식별자**): a fundamental abstraction mechanism used to denote language entities or constructs
  - Variables, procedures, functions, constants can have names assigned by programmers.
- Fundamental step in describing semantics is to describe the **conventions** that determine the meaning of each name.
- Most languages include concepts of location and value
  - **Location**: place where value can be stored; usually a relative location
  - **Value**: any storable quantities

Name and Attributes (cont'd)

- **Attributes**: properties that determine **the meaning of the name** to which they are **associated**
- Example in C:

```
const int n = 5;
```

  - Attributes: data type (integer constant) and value (5)
- Example in C:

```
double f(int n) {
…
}
```

  - Attributes: "function," number, names, data type of parameters, return value data type, body of code to be executed

Name and Attributes (cont'd)

- **Assignment** statements associate attributes to names
- Example: `x = 2;`
  - Associates attribute "value 2" to variable x
- Example in C++:

```
int* y;
y = new int;
```

  - pointer variable
  - Allocates memory (associates location to *y)
  - Associates value to y

Binding

- **Binding**: associating attributes to names via
  - declarations (e.g., `int x;`)
  - assignments (e.g., `int x =2;`)
  - function declarations (prototype, e.g., `double f (int);`) and function definition (e.g., specifying the code of `f`)
- **The bindings can be explicit** (by programmers) **or implicit** (auto)

  e.g. in c, `int x;`
  - explicit binding: the **data type** of `x`
  - implicit binding: the **location** of `x`

  e.g. in python, `x = 5`
  - entire declaration are implicit

Binding Time

- **Binding Time: the time when an attribute is bound to a name.**
  - **Static binding** (static attribute):

    occurs before execution
    - language definition/implementation time: e.g., the range of `int`
    - translation time (parsing/semantic analysis): e.g., the data type of a variable
    - link time: the body of external function
    - load time: the location of global variable
  - **Dynamic binding** (dynamic attribute):

    occurs during execution e.g.,
    - entry/exit from procedure or program
    - method overriding in OO

The Symbol Table, Environment, Memory

- A translator creates a **data structure** to maintain bindings
  - The **Symbol Table :** can be thought of as a function that expresses the **binding of attributes to names**



- During the execution of a compiled program, attributes (locations and values) are also maintained
  - **Environment**: the bindings of names to storage location
  - **Memory**: the bindings of storage locations to values





Lexical, Syntax, Semantic Analysis

- Parsing phase of translation includes three types of analysis:
  - Lexical analysis: determines whether a string of characters represents a token
  - Syntax analysis: determines whether a sequence of tokens represents a phrase in the context-free grammar
  - Static semantic analysis: establishes attributes of names in declarations and ensures that the use of these names conforms to their declared attributes

Declarations, Blocks, and Scope

Where can declarations happen?

- Blocks: a sequence of declarations followed by statements
  - { }, begin-end, …
  - Algol descendants: C/C++, Java, Pascal, Ada, …

  e.g., C
  - Function body
  - Anywhere a statement can appear (compound statement)
- External/global
- Structured data type
- Class

C++ Example

```
const int Maximum = 100;
struct FullName {string Lastname, string FirstName};

class Student {
private:
     struct FullName name;
      int Age;
public:
     void setValue(const int a, struct FullName name);
      int TStudent();

     …
};

void Student::setAge(const int a, string lName, string fName) {
      int i;
     Age = a;
     {    int j;
          name.LastName = lName;
          name.FirstName = fName;
     }
}
```

Scope of Binding

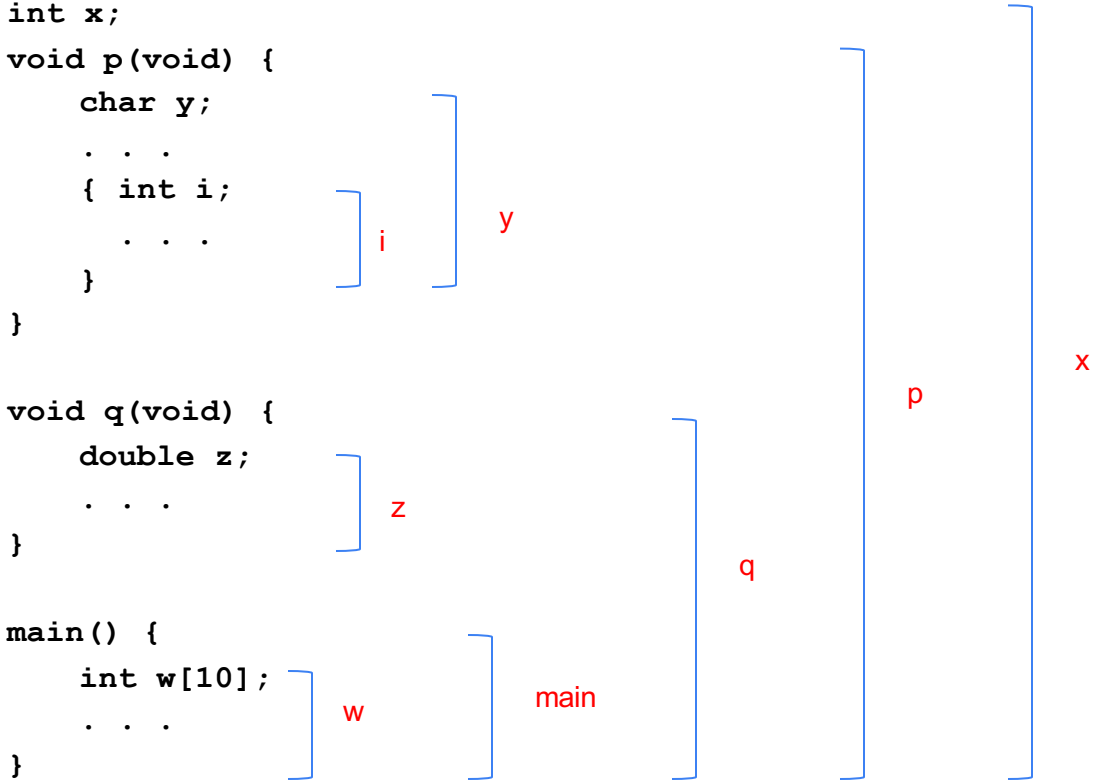
- **Scope of Binding**: the region of the program where the binding is maintained (is valid, applies).
- **Block-structured language**
  - *lexical scope (static scope)*: from the declaration to the end of the block containing the declaration.
  - *dynamic scope* : If symbol table is managed dynamically (during execution), declarations will be processed as they are encountered along an execution path

Example: Scope of Binding

```
int x;
void p(void) {
    char y;
    . . .
    { int i;
      . . .
    }
}

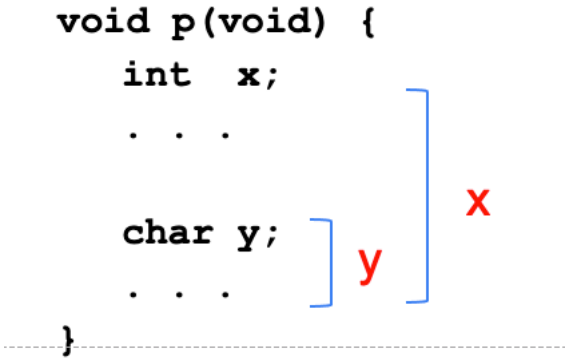
void q(void) {
    double z;
    . . .
}

main() {
    int w[10];
    . . .
}
```

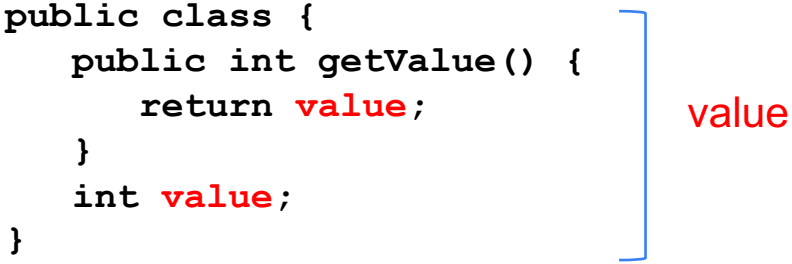


Declaration Before Use

- **Declaration before use rule**:
  - in C, scope of a declaration extends from the point of declaration to the end of the block in which it is located



- Exception in OO languages:
  - Scope of local declarations inside a class declaration includes the whole class.



Scope Hole

- Declarations in nested blocks take precedence over previous declarations. That is, binding becomes invisible/hidden.
- A *global variable* is said to have a **scope hole** in a block containing a local declaration with the same name
  - Use scope resolution operator **::** in C++ to access the global variable
- Local declaration is said to **shadow** its global declaration
- **Visibility**: includes only regions where the bindings of a declaration apply

Scope Hole

```
int x;

void p(void) {
   char x;
   x = 'a';
   . . .
}

main() {
   x = 2;
   . . .
}
```

x (bound with character data type)

scope hole

x (bound with integer data type)

Access Hidden Declarations

- scope resolution operator :: (C++)

```
int x;

void p(void) {
    char x;
    x = 'a';
    ::x = 42;
    . . .
}

main() {
    x = 2;
    . . .
}
```

x (bound with character data type)

the hidden integer variable x

x (bound with integer data type)

Hide a Declaration

- File 1 in C:

extern int x;

File 2 in C:

int x;

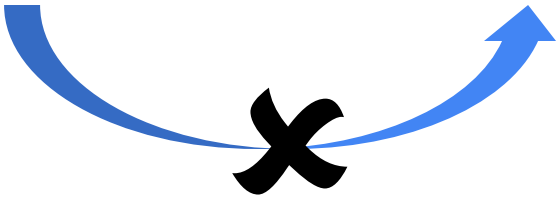


- File 1 in C:

extern int x;

File 2 in C:

static int x;



Scopes Rule

- **Scope rules** need to be constructed such that **recursive** (self-referential) declarations are possible when they make sense
  - Example: **functions** must be allowed to be **recursive**, so function name must have scope beginning **before the block of the function body**

```
int factorial (int n) {
    /* scope of factorial begins here */
    /*factorial can be called here */
    …
}
```

Scope Rules (cont'd)

- `int x = x + 1; /*.  ???  *?`
  - scope of x begins '='

- Ada: self-reference in variable declarations are outlawed.
- Java: the self-reference is flagged as an uninitialized use.
- C/C++:
  - not an error if x is a local variable
  - simply add one to the uninitialed value of x

The Symbol Table

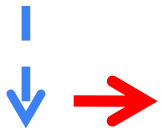
- Symbol Table:
  - Must support insertion, lookup, and deletion of names with associated attributes, representing binding in declarations
- A lexically scoped, block-structured language requires a stack-like data structure to perform scope analysis:
  - On block entry: all declarations of that block are processed and bindings added to symbol table
  - On block exit: bindings are removed, restoring any previous bindings that may have existed



Static vs. Dynamic Scope

- Static scope (lexical scope):
  - scope maintained statically (during **compilation**)
  - follow **the layout of source codes**
  - **used in most languages**
- Dynamic scope:
  - scope maintained dynamically (during **execution**)
  - follow **the execution path**
  - few languages use it (The bindings cannot be determined statically, may depend on user input).
    - Lisp: considered a bug by its inventor.
    - Perl: can choose lexical or dynamic scope

Static Scope

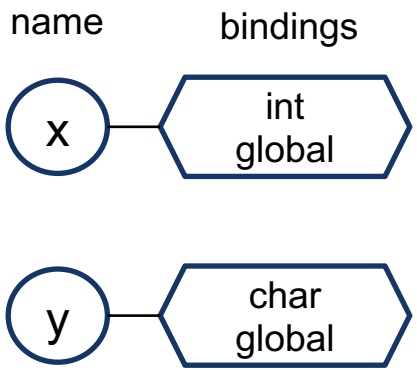


```
int x = 1;
char y = 'a';

void p(void) {
    double x=2.5;
    printf("%c\n",y)
    ;
}
void q(void) {
    int y = 42;
    printf("%d\n",x)
    ;
    p();
}

main() {
    char x = 'b';
    q();
}
```

The symbol table and the bindings



Static Scope

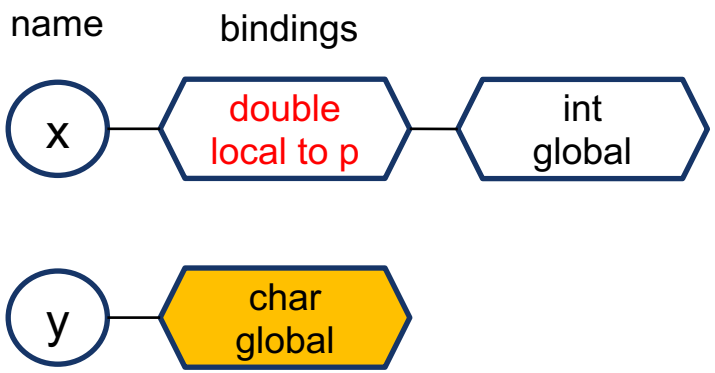
```
int x = 1;
char y = 'a';

void p(void) {
    double x=2.5;
    printf("%c\n",y);
}

void q(void) {
    int y = 42;
    printf("%d\n",x);
    p();
}

main() {
    char x = 'b';
    q();
}
```

The symbol table in **p**:
the bindings available in **p**



Static Scope



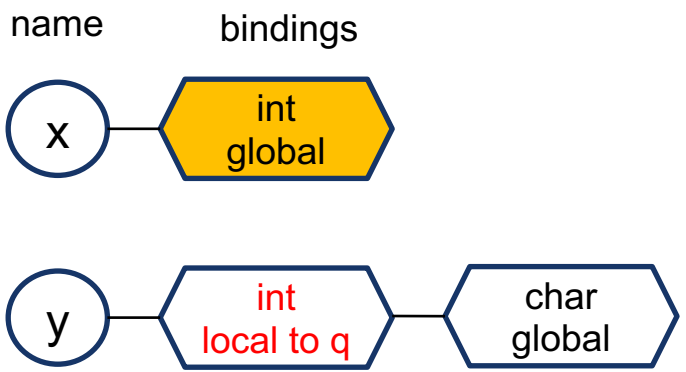
```
int x = 1;
char y = 'a';

void p(void) {
    double x=2.5;
    printf("%c\n",y);
}

void q(void) {
    int y = 42;
    printf("%d\n",x);
    p();
}

main() {
    char x = 'b';
    q();
}
```

The symbol table in **q**:
the bindings available in **q**

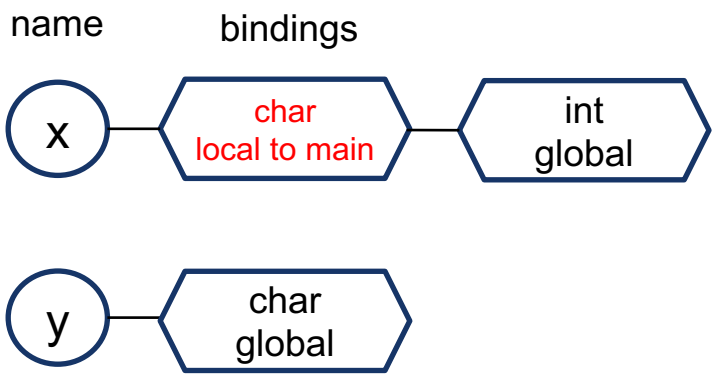


Static Scope

```
int x = 1;
char y = 'a';

void p(void) {
    double x=2.5;
    printf("%c\n",y);
}
void q(void) {
    int y = 42;
    printf("%d\n",x);
    p();
}
main() {
    char x = 'b';
    q();
}
```

The symbol table in **main**:
the bindings available in **main**



Static Scope

- The symbol table in previous slides are built **during compilation**
- The bindings are used in generating the machine code
- Result:

```
1 // x: int global (1)
a // y: char global ('a')
```

- why?

[HW] Practice for Static Scope

Point 1 

Point 2 

Point 3 

```
int x,y;

void g(void) {
    x = x + 1;
    y = x + 1;
}

void f(void) {
    int x;
    y = y + 1;
    x = y + 1;
    g();
}

main() {
    x = 1;
    y = 2;
    f();
    g();
    printf("x=%d,y=%d\n",x,y);
}
```

Question 1:

Draw the symbol table at the given points in the program, using static scope?

Question 2:

What does the program print, using static scope?

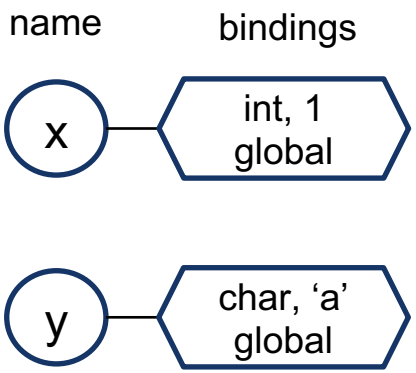
What if dynamic scope is used?

```
int x = 1;
char y = 'a';

void p(void) {
    double x=2.5;
    printf("%c\n",y);
}

void q(void) {
    int y = 42;
    printf("%d\n",x);
    p();
}

main() {
    char x = 'b';
    q();
}
```



What if dynamic scope is used?

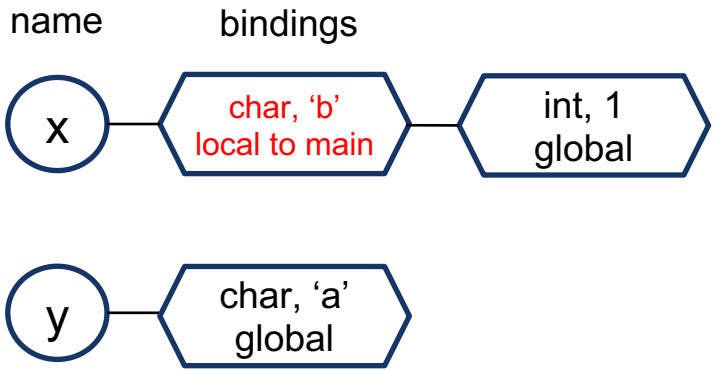
```
int x = 1;
char y = 'a';

void p(void) {
    double x=2.5;
    printf("%c\n",y);
}

void q(void) {
    int y = 42;
    printf("%d\n",x);
    p();
}

main() {
    char x = 'b';
    q();
}
```

The symbol table in **main**:
the bindings available in **main**

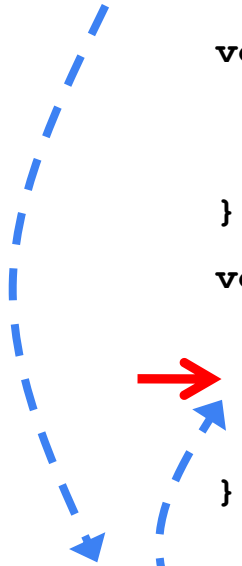


What if dynamic scope is used?

```
int x = 1;
char y = 'a';

void p(void) {
    double x=2.5;
    printf("%c\n",y);
}
void q(void) {
    int y = 42;
    printf("%d\n",x);
    p();
}

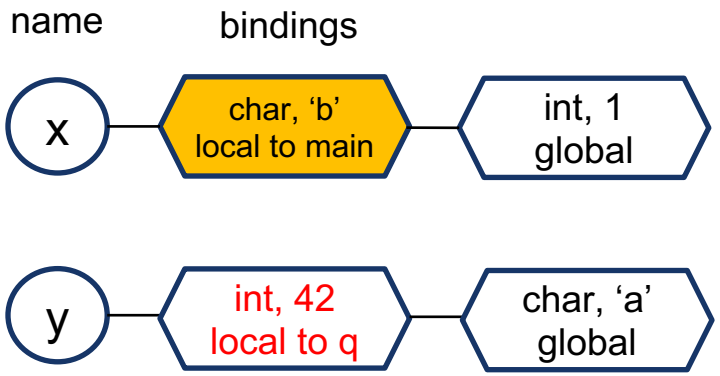
main() {
    char x = 'b';
    q();
}
```



98

b: ascii 98

The symbol table in **q**:
the bindings available in **q**



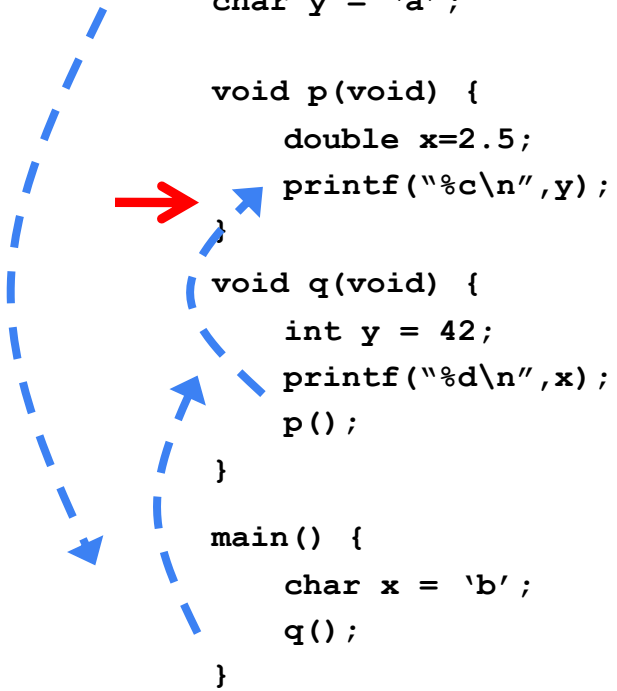
What if dynamic scope is used?

```
int x = 1;
char y = 'a';

void p(void) {
    double x=2.5;
    printf("%c\n",y);
}

void q(void) {
    int y = 42;
    printf("%d\n",x);
    p();
}

main() {
    char x = 'b';
    q();
}
```

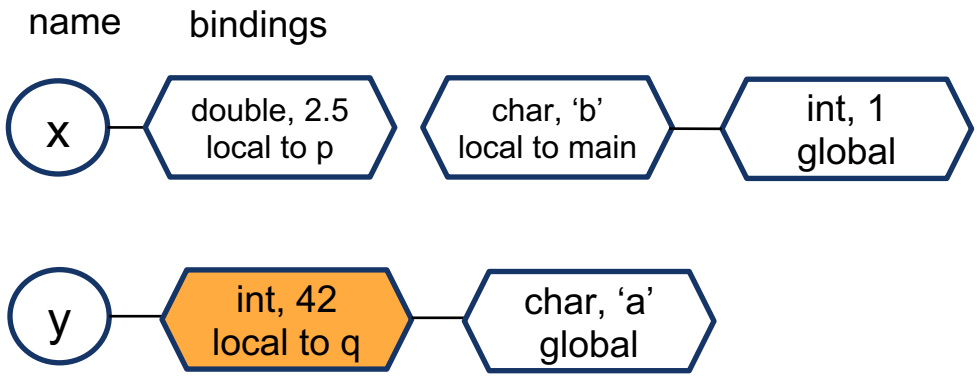


```
98

*
```

b: ascii 98
*: ascii 42

The symbol table in **p**:
the bindings available in **p;**



[HW] Practice for Dynamic Scope

Point 1

Point 2

Point 3

```
int x,y;

void g(void) {
    x = x + 1;
    y = x + 1;
}

void f(void) {
    int x;
    y = y + 1;
    x = y + 1;
    g();
}

main() {
    x = 1;
    y = 2;
    f();
    g();
    printf("x=%d,y=%d\n",x,y);
}
```

Question 1:

Draw the symbol table at the given points in the program, using **dynamic** scope?

Question 2:

What does the program print, using **dynamic** scope?

The Symbol Table for struct

- Two implications for struct variables:
  - A **struct** declaration actually **contains a local symbol table** itself as an attribute
  - This local symbol table cannot be deleted until the struct variable itself is deleted from the global symbol table of the program

```
(1)  struct{
(2)      int a;
(3)      char b;
(4)      double c;
(5)  } x = {1, 'a',2.5};

(6)  void p(){
(7)      struct{
(8)          double a;
(9)          int b;
(10)         char c;
(11)     } y = {1.2,2, 'b'};
(12)     printf("%d, %c, %g\n",x.a,x.b,x.c);
(13)     printf("%f, %d, %c\n",y.a,y.b,y.c);
(14) }

(15) main(){
(16)     p();
(17)     return 0;
(18) }
```

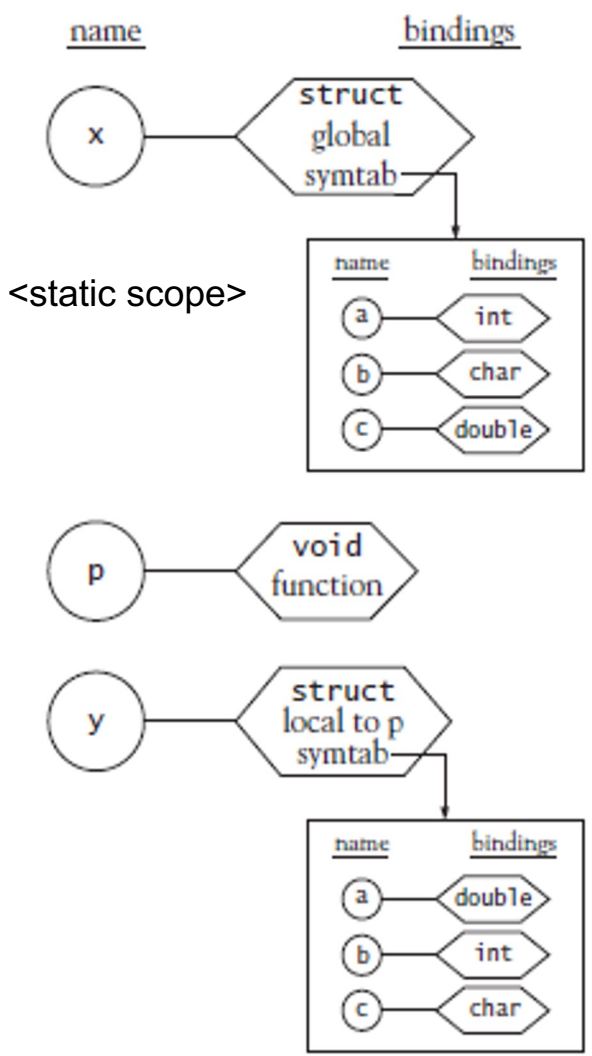


Figure 7.17 Code example illustrating scope of local declarations in a C `struct`

The Symbol Table for Reference

- Any scoping structure that can be referenced directly must also have **its own symbol table**
- Examples:
  - Classes, structs, and namespaces in C++
  - Classes and packages in Java
  - Named scopes in Ada
- Typically, there will be a table for each scope in a **stack of symbol tables**
  - When a reference to a name occurs, a search begins in the current table and continues to the next table if not found, and so on

Name Resolution and Overloading

Name Resolution (이름 결정) and Overloading

- Addition operator + : at least two different operations
  - *integer* addition and *floating-point* addition
  - + operator is said to be **overloaded**
- Translator must look at **the data type of each operand to determine** which operation is indicated
- **Overload resolution**: process of choosing a unique function among many with the same name
  - Lookup operation of a symbol table must search on **name** plus **number** and **data type of parameters**

Name Resolution and Overloading

- Consider these function calls:

```
max(2,3);       //max #1
max(2.1,3.2);   //max #2
max(1,3,2);     //max #3
```

- Symbol table can determine the appropriate function based on the number and type of parameters
- **Calling context**: the information contained in each call
- But this **ambiguous** call depends on the language rules (if any) for converting between data types

```
max(2.1,3);   // which max?
```

```
int max(int x, int y){ // max #1
    return x > y ? x : y;
}

double max(double x, double y){ // max #2
    return x > y ? x : y;
}

int max(int x, int y, int z){ // max #3
    return x > y ? (x > z ? x : z) : (y > z ? y : z);
```

Figure 7.21 Three overloaded max functions in C++

Name Resolution and Overloading

- Adding these definitions in C++ and Ada:

  `max(2.1,3);//max #5`
- Different conversion rules for not perfect match
  - C++: automatic conversions from/to any direction
  - Java: conversions allowed in certain directions (w/o loosing information. e.g., int -> double).
  - Ada: automatic conversions not allowed.

```
double max(int x, double y){ // max #4
    return x > y ? (double) x : y;
}

double max(double x, int y){ // max #5
    return x > y ? x : (double) y;
}
```

Figure 7.22 Two more overloaded `max` functions in C++ (see Figure 7.21)

Name Resolution and Overloading

- Additional information in a **calling context** may be used for overload resolution:
  - Ada: the return type and names of parameters to be used for overload resolution
  - C++ and Java: ignore the return type
- Ada and C++ (but not Java): allow built-in operators to be overloaded
- When overloading a built-in operator, we must accept its syntactic properties
  - Example: cannot change the associativity or precedence of the + operator

[HW] Overload Resolution Example

1: int sum(int, int);
2: double sum(double, int);
3: double sum(double, double);
0: error

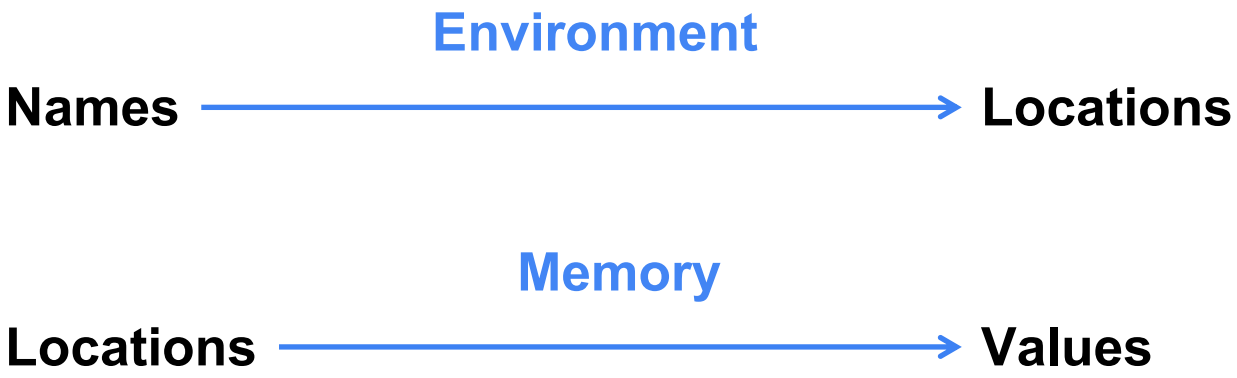
int x;
double y;

	C++	Java	Ada
x = sum(3, 4);			1
y = sum(3, 4);			0
x = sum(3, 4.5);			0
y = sum(3, 4.5);			0
x = sum(3.5, 4);			0
y = sum(3.5, 4);			2
x = sum(3.5, 4.5);			0
y = sum(3.5, 4.5);			3

Allocation, Lifetimes and the Environment

Environment and Memory

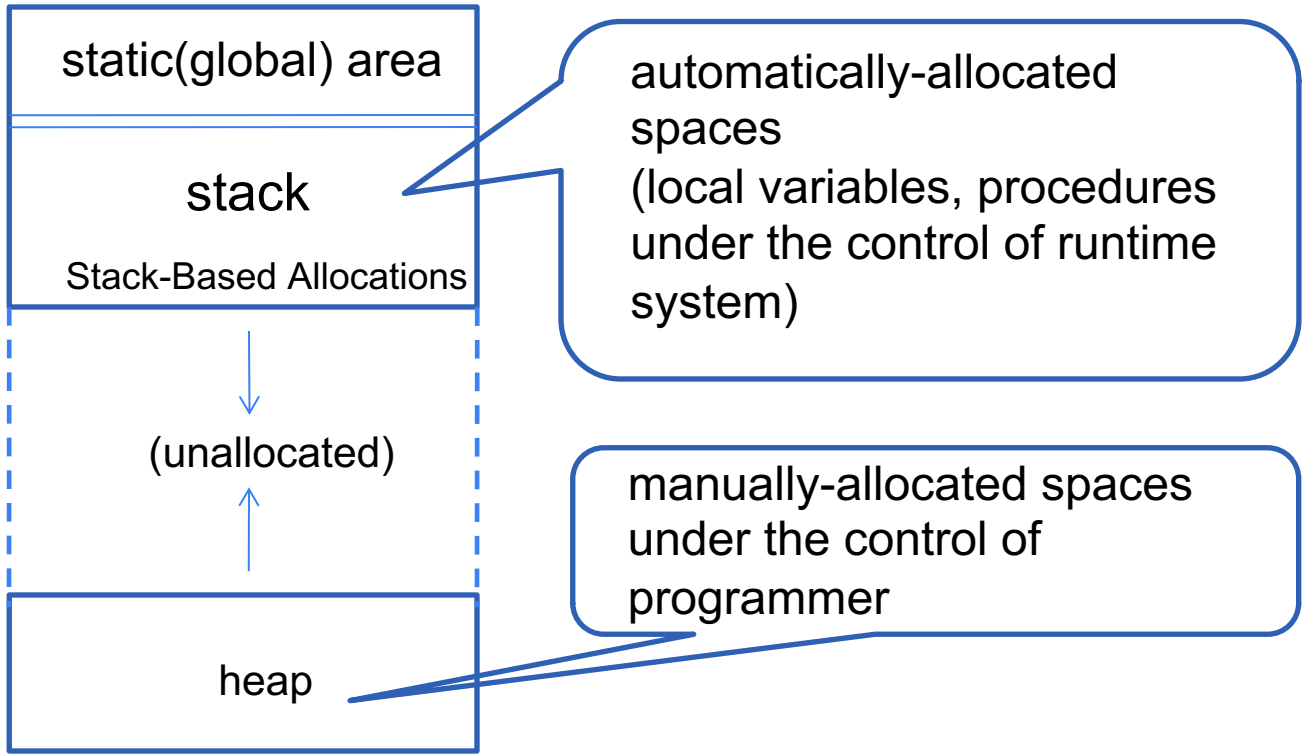
- During the execution of a compiled program, attributes (locations and values) are also maintained
- **Environment**: the bindings of names to storage location
- **Memory**: the bindings of storage locations to values



Environment

- **Location:** one specific attribute of names
- **Environment:** maintain bindings of names to locations
  - Constructed statically (at load time), dynamically (at execution time), or with a mixture of both
- **Static vs. dynamic allocation**
  - FORTRAN: completely static allocation
  - LISP: completely dynamic allocation
  - Algol-descendants (C, C++, Ada, Java) : combination
    - global variables: static allocation
    - local variables: dynamic allocation

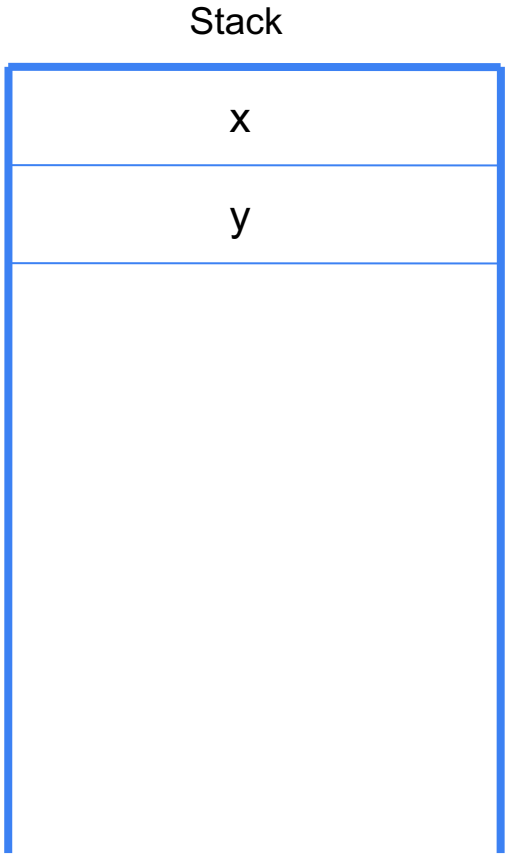
Allocations



Example

```
A: {  int   x;
      char  y;
   B: {  double  x;
         int     a;
      }
   C: {  char    y;
         int     b;
      D: {  int     x;
            double  y;
         }
      }
}
```

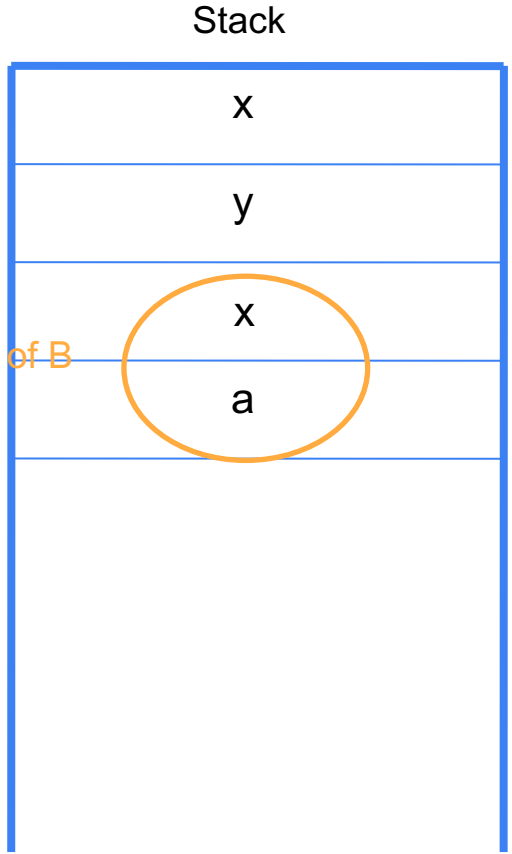




Example

```
A: {  int   x;
      char  y;
   B: {  double  x;
         int     a;
      }
   C: {  char    y;
         int     b;
      D: {  int     x;
            double  y;
         }
      }
}
```

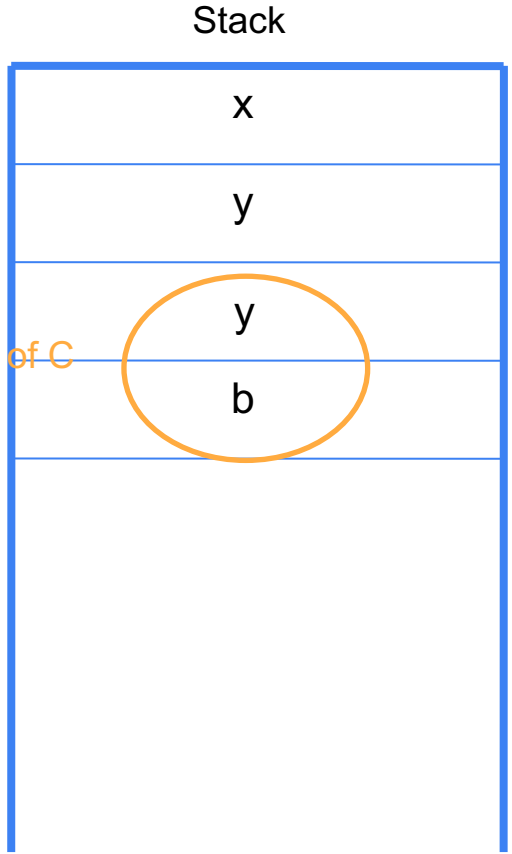
Lifetime is the duration of B



Example

```
A: {  int   x;
      char  y;
   B: {  double  x;
         int     a;
      }
   C: {  char    y;
         int     b;
      D: {  int     x;
            double  y;
         }
      }
}
```

Lifetime is the duration of C

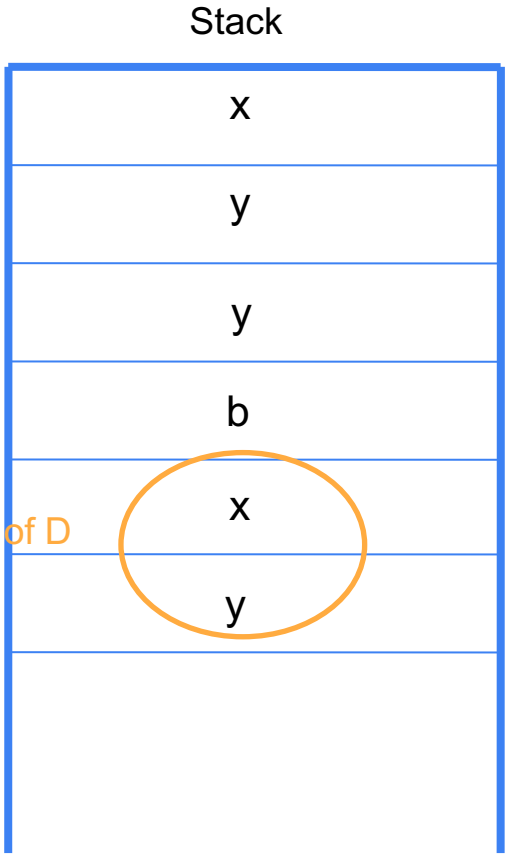


Example

```
A: {  int   x;
      char  y;
   B: {  double  x;
         int     a;
      }
   C: {  char    y;
         int     b;
     D: {  int     x;
           double  y;
         }
      }
}
```



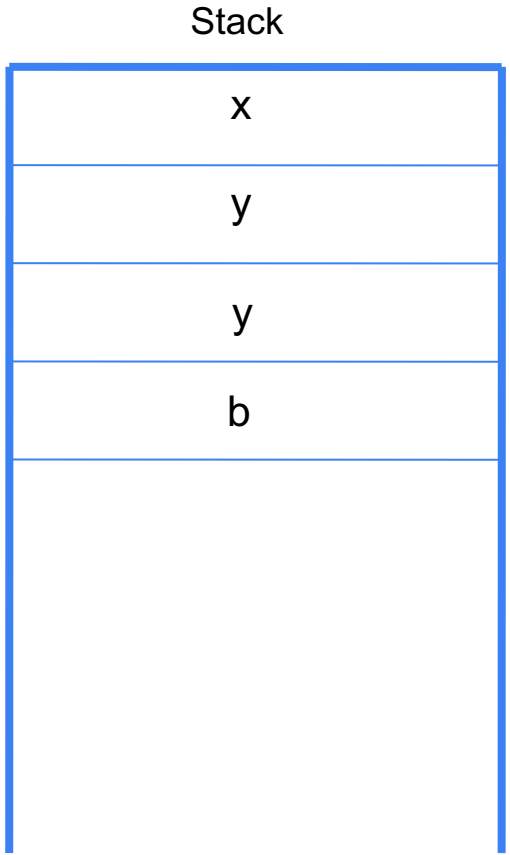
Lifetime is the duration of D



Example

```
A: {  int   x;
      char  y;
   B: {  double  x;
         int     a;
      }
   C: {  char    y;
         int     b;
      D: {  int     x;
            double  y;
         }
      }
}
```





Allocation, Lifetimes, and the Environment (cont'd.)

- Memory for local variables within a function: **not be allocated before a function call**
- **Activation**: a call to a function
- **Activation record (Stack frame)**: the corresponding region of allocated memory
- In a block-structured language with lexical scope, the same name can be associated with different locations, but **only one of these can be accessed at any one time**
- **Lifetime** (or **extent**) of an object is the duration of its allocation in the environment
  - Lifetime extends through a scope hole

Heap-Based Allocation

- **Heap**: an area in memory from which locations can be allocated in response to calls to `new` (or malloc in c)
- **Dynamic allocation**: allocation on the heap
- **Pointer**: an object whose stored value is a reference to another object
- C allows the initialization of pointers that do not point to an allocated object:
  - Objects must be manually allocated by use of an allocation routine
  - Variable can be dereferenced using the unary * operator

Heap-Based Allocation (cont'd)

- Many languages require that heap deallocation be managed automatically (e.g., Java)
- Heap allocation/deallocation and explicit pointer manipulation are inherently *unsafe* operations
  - Can introduce seriously faulty runtime behavior that may even compromise the operating system
- **Storage class**: the type of allocation
  - Static (for global variables)
  - Automatic (for local variables) -- stack
  - Dynamic (for heap allocation)

Heap-Based Allocation

- **C**

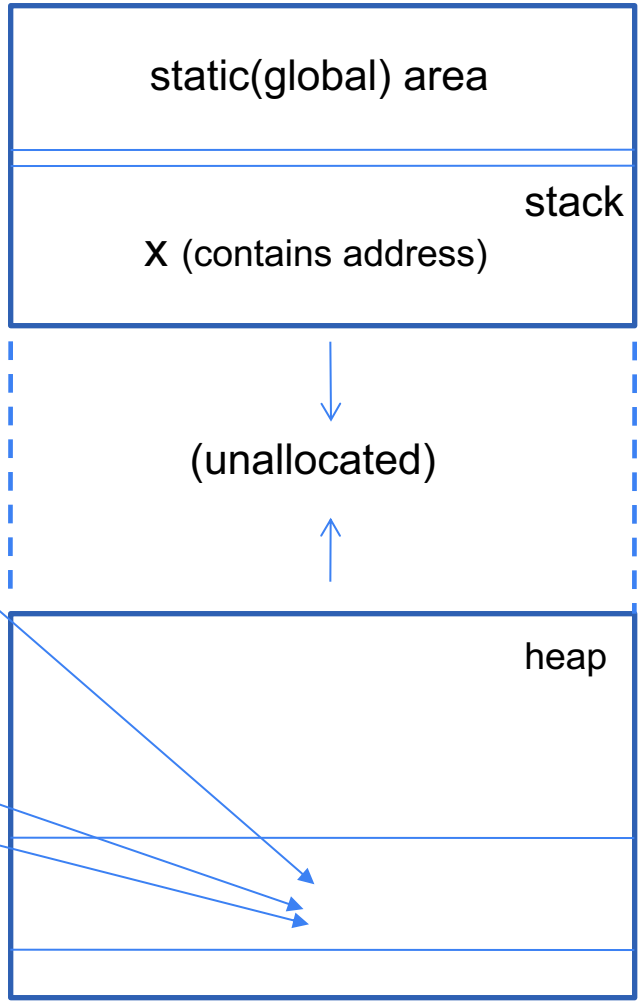
```
int *x;
x= (int *)malloc(sizeof(int));
free(x);
```

- **C++**

```
int *x;
x= new int;
delete x;
```

- **Java**

```
Integer x = new Integer(2);
//no delete
//need garbage collection)
```



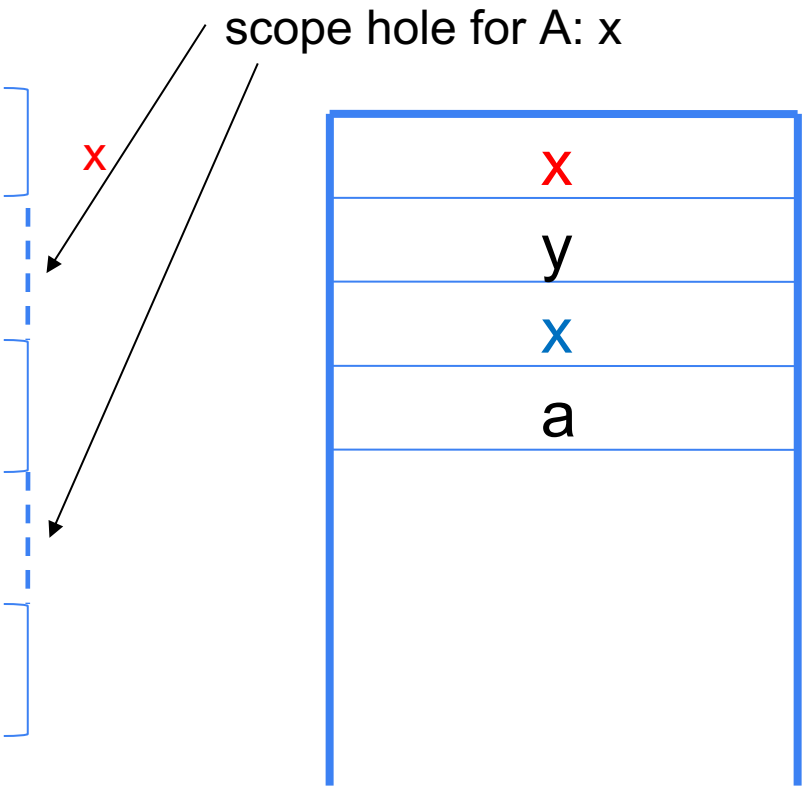
Scope vs. Lifetime

- Lifetime beyond scope:
  - alive in scope hole
  - alive outside scope

- Scope beyond lifetime (unsafe)

Example: Alive in Scope Hole

```
A: {   int    x;
       char   y;
    B: {   double   x;
           int      a;
       }
    C: {   char     y;
           int      b;
      D: {   int      x;
             double   y;
           }
       }
}
```



Example: Alive outside scope

```
int func(void) {
   static int counter = 0;
   counter += 1;
   return counter;
}

main(){
   int i;
   int x;
   for (i=0; i<10; i++)
     { x=func(); }
   printf("%d\n", x);
}
```

counter

Example: Scope beyond Lifetime

Dangling pointer:

```
int *x, *y, *z;

x = (int *) malloc(sizeof(int));
*x = 2;
y = x;
free(x);

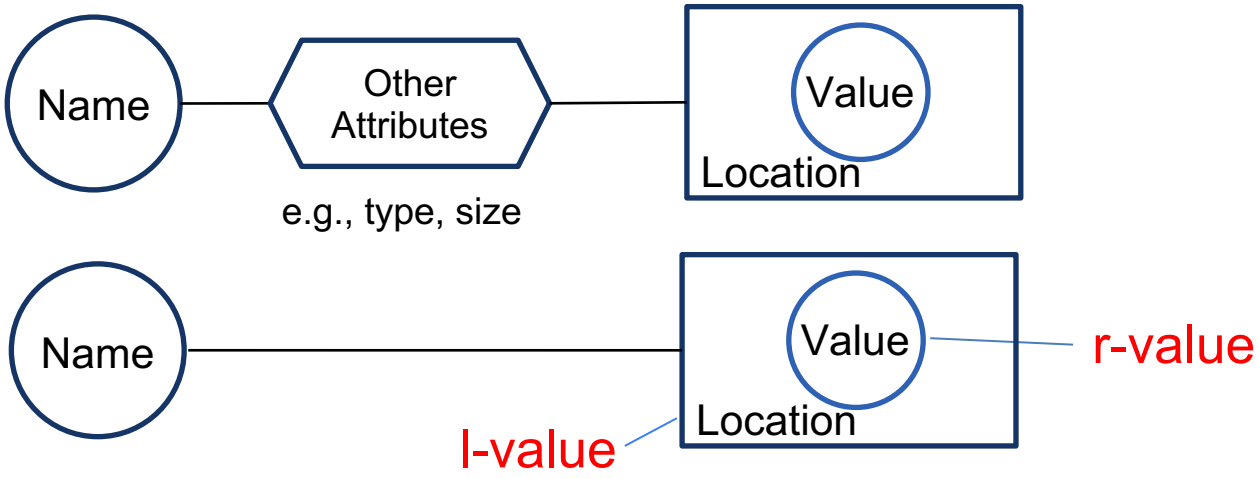
. . .

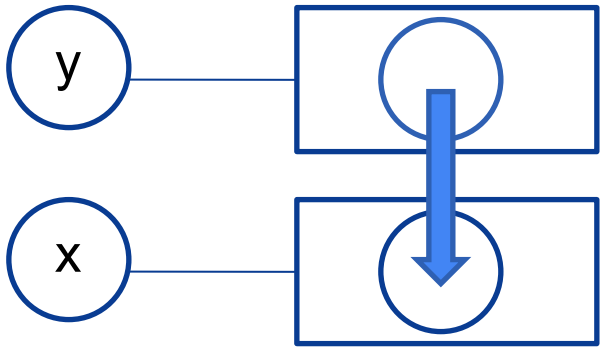
printf("%d\n",*y);
```

Variable

- **Variable**: an object whose stored value can change during execution
  - Is completely specified by its attributes (name, location, value, data type, size of memory storage)
- **l-value** : variable on left side of assignment statement (**x** = 4)
- **r-value** : variable on right side of assignment statement (x = **4**)
- **Address of operator** (&) in C: turns a reference into a pointer to fetch the address of a variable
- **Assignment by sharing**: the **location** is copied instead of the value
- **Assignment by cloning**: allocates **new location**, copies value, and binds to the new location
- Both are sometimes called **pointer semantics** or **reference semantics**

Box-and-Circle Diagram for Variables





Assignment by cloning:

x = y

Assignment by sharing

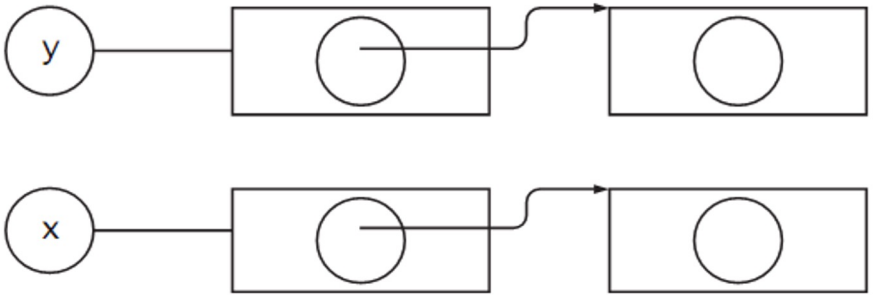


Figure 7.40 Variables as implicit references to objects

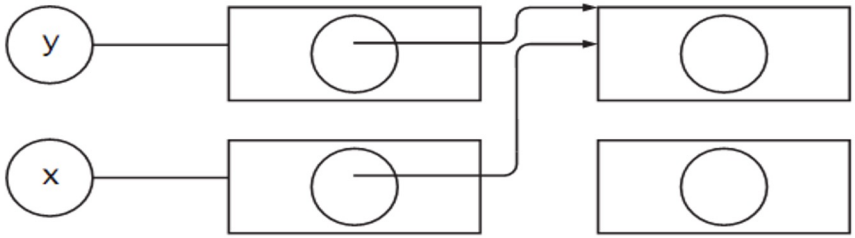


Figure 7.41 Assignment by sharing of references

garbage

(4) x = y;

Java (**reference/pointer semantics**):

(1) Student x = new Student("Amy");

(2) Student y = new Student("John");

(3) x.setAge(19);

(4) x = y;

- **assignment by sharing**: the location is copied instead of the value

Assignment by Cloning (Deep Copy)

- **Assignment by cloning**: allocates new location, copies value, and binds to the new location

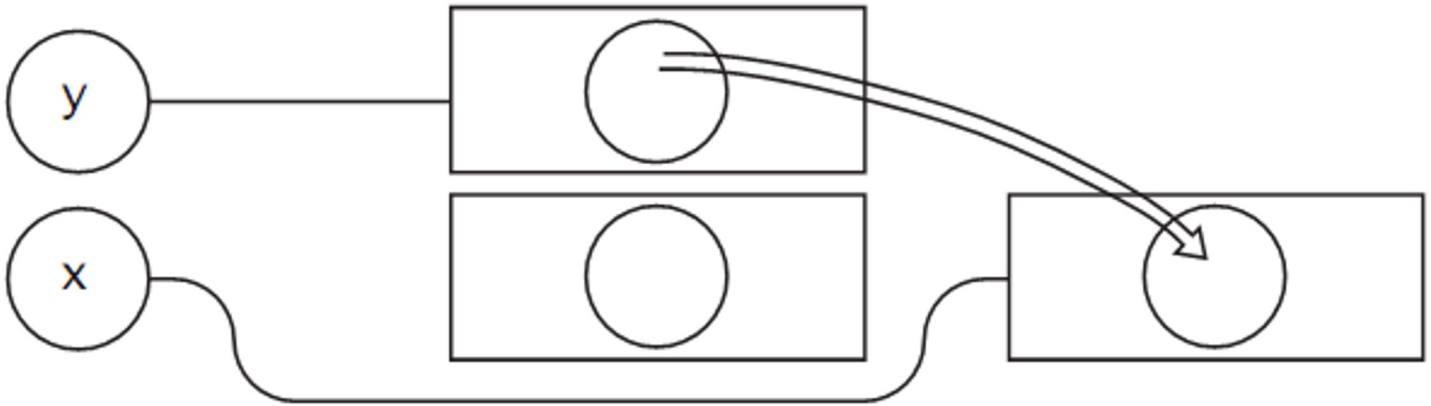


Figure 7.39 The result of assignment by cloning

Constants

- **Constant**: an entity with a fixed value for the duration of its existence in a program
  - no location attribute
  - Sometimes say that a constant has **value semantics**
- **Literal**: a representation of characters or digits
- **Compile-time constant**: its value can be computed during compilation
- **Static constant**: its value can be computed at **load time**
- **Manifest constant**: a name for a **literal**
- **Dynamic constant**: its value must be computed **during execution**

Constants (cont'd.)

- `a`, `b`: compile-time constants
  - `a`: manifest constant (literal)
- `c`: static (load-time constant)
- `d`: dynamic constant

```
#include <stdio.h>
#include <time.h>

const int a = 2;
const int b = 27+2*2;
/* warning - illegal C code! */
const int c = (int) time(0);

int f( int x){
    const int d = x+1;
    return b+c;
}
...
```

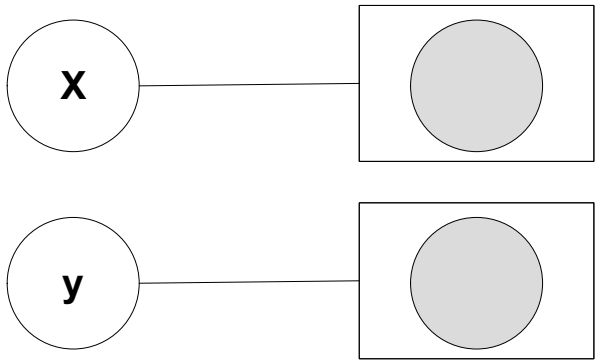
Aliases, Dangling References and Garbage

Aliases

- **Alias**: occurs when the same object is bound to different names at the same time
- Can occur during procedure call, through the use of pointer variables, or through assignment by sharing

```
(1) int *x, *y;
(2) x = (int *)malloc(sizeof(int));
(3) *x = 1;
(4) y = x;
(5) *y = 2;
(6) printf("%d\n",*x);
```

After line 1:



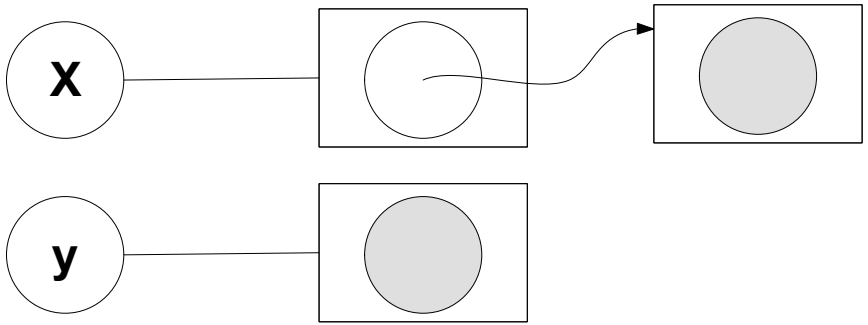


: uninitialized value

Aliases

```
(1) int *x, *y;
(2) x = (int *)malloc(sizeof(int));
(3) *x = 1;
(4) y = x;
(5) *y = 2;
(6) printf("%d\n",*x);
```

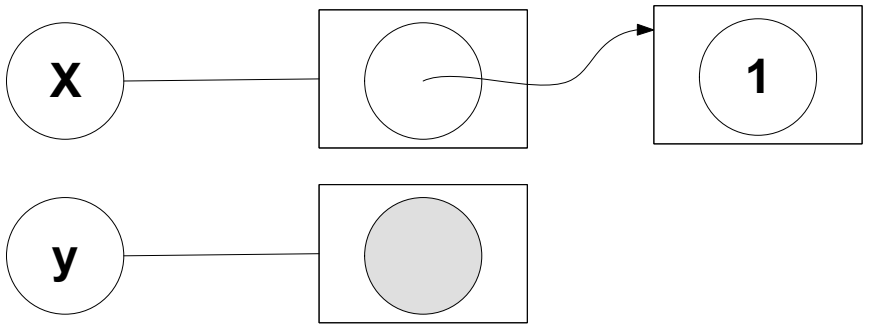
After line 2:



Aliases

```
(1) int *x, *y;
(2) x = (int *)malloc(sizeof(int));
(3) *x = 1;
(4) y = x;
(5) *y = 2;
(6) printf("%d\n",*x);
```

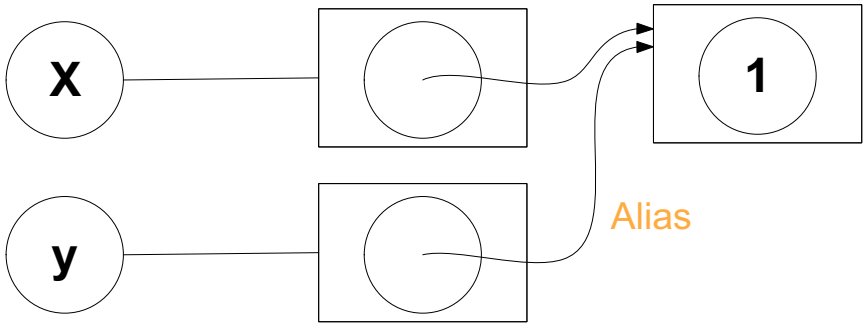
After line 3:



Aliases

```
(1) int *x, *y;
(2) x = (int *)malloc(sizeof(int));
(3) *x = 1;
(4) y = x; //alias 발생
(5) *y = 2;
(6) printf("%d\n",*x);
```

After line 4:

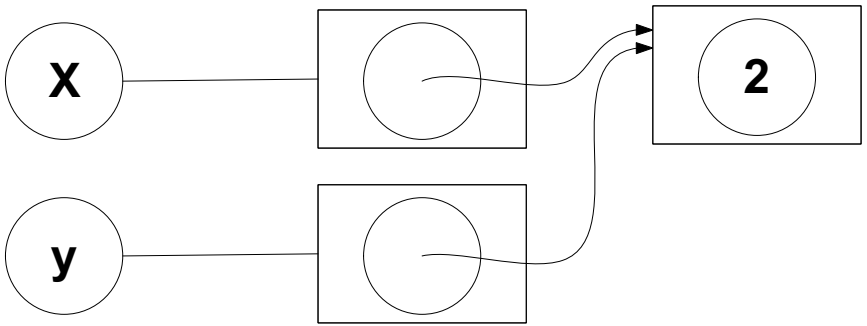


Aliases

```
(1) int *x, *y;
(2) x = (int *)malloc(sizeof(int));
(3) *x = 1;
(4) y = x;
(5) *y = 2;
(6) printf("%d\n",*x);
```



After line 5:



Alias

- Aliases can potentially cause harmful side effects
- **Side effect**: any change in a variable's value that persists beyond the execution of the statement
  - Not all side effects are harmful; an assignment statement is intended to cause one
- Side effects that change variables whose names do not directly appear in the statement are potentially harmful
  - Cannot be determined from the written code
- **Aliasing due to pointer assignment is difficult to control**

[HW] Practice for Aliases

```
(1) #include <stdio.h>
(2)  main(){
(3)    int **x;
(4)    int *y;
(5)    int z;
(6)    x = (int**)malloc(sizeof(int*));
(7)    y = (int*)malloc(sizeof(int));
(8)    z = 1;
(9)    *y = 2;
(10)   *x = y;
(11)   **x = z;
(12)   printf("%d\n",*y);
(13)   z = 3;
(14)   printf("%d\n",*y);
(15)   **x = 4;
(16)   printf("%d\n",z);
(17)   return 0;
(18) }
```

Question 1:

Draw box-and-circle diagrams of the variables after line 11 and 15.

Question 2:

Which variables are aliases at each of those points?

Question 3:

What does the program print?

after line 11:

after line 15:



Dangling References

- **Dangling reference**:

  a location that has been deallocated from the environment but can still be accessed by a program

  – Occurs when a pointer points to a deallocated object

```
int *x, *y;
…
x = (int *)malloc(sizeof(int));
…
*x = 2;
…
y = x;
free(x);
/* *y is now a dangling reference
*/
…
printf("%d\n",*y); /*illegal
reference*/
```

Dangling References

- Can result from automatic deallocation of local variables on exit from a block, with the C address of operator

```
{
  int *x;
  {
     int y;
     y = 2;
     x = &y;
  }
/* *x is now a dangling
reference */
}
```

Dangling References

```
int* dangle(void)
{
  int x;
  return &x;
}
…
y = dangle();
/* *y is a dangling reference */
```

Garbage

- **Garbage**: memory that has been allocated in the environment but is now inaccessible to the program
- Can occur in C by **failing to call free before reassigning** a pointer variable

```
int *x;
…
x = (int *) malloc(sizeof(int));
x = 0;
```

Garbage (cont'd)

- A program with dangling references may:
  - Produce incorrect results
  - Corrupt other programs in memory
  - Cause runtime errors that are hard to locate
- For this reason, it is useful to remove the need to deallocate memory explicitly from the programmer
- **Garbage collection**: process of automatically reclaiming garbage
- Language design is a key factor in the kind of runtime environment necessary for correct execution of programs

References

[1] Programming languages principles and practice, Louden & Lambert, 3rd edition, Course Technology